ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Задачі на графах

Звіт про виконання лабораторної роботи № 7

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Вінниця 2026

Тема: Задачі на графах. Пошук ваги мінімального кістяка неорієнтованого зваженого зв'язкового графа за алгоритмом Краскала.

Мета: застосувати алгоритми на графах для розв'язку олімпіадних задач з програмування; реалізувати на мові C++ алгоритм Краскала та систему неперетинних множин (DSU) для знаходження ваги мінімального кістяка графа; оцінити ефективність реалізованого алгоритму.

ЗАВДАННЯ

1. Реалізувати програму для визначення ваги мінімального кістяка (MST) неорієнтованого зваженого зв'язкового графа згідно з варіантом 12.

2. Реалізувати зчитування вхідних даних з файлу input.txt (кількість вершин N <= 100, кількість ребер M <= 6000 та M трійок чисел A, B, C — номери вершин і вага ребра C <= 30000) та запис результату у файл output.txt.

3. Реалізувати систему неперетинних множин (DSU) зі стисненням шляху (find) та об'єднанням за рангом (unite) для перевірки компонент зв'язності.

4. Реалізувати алгоритм Краскала: сортування ребер за зростанням ваги та послідовний вибір N - 1 найлегших ребер, що не утворюють циклів.

5. Передбачити автоматичну генерацію випадкового зв'язкового зваженого графа у файл input.txt при передачі аргументів командного рядка або за відсутності вхідного файлу.

КОД ПРОГРАМИ

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <random>
#include <set>
#include <cstdlib>

using namespace std;

struct Edge {
    int u;
    int v;
    int w;
};

struct DSU {
    vector<int> parent;
    vector<int> rnk;

    DSU(int n) : parent(n + 1), rnk(n + 1, 0) {
        for (int i = 0; i <= n; i++) parent[i] = i;
    }

    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (rnk[a] < rnk[b]) swap(a, b);
        parent[b] = a;
        if (rnk[a] == rnk[b]) rnk[a]++;
        return true;
    }
};

void generateRandomGraph(int n, int m) {
    if (n < 1) n = 1;
    if (m < n - 1) m = n - 1;

    int maxEdges = (n > 1) ? n * (n - 1) / 2 : 0;
    if (m > maxEdges) m = maxEdges;

    mt19937 gen(12345);
    uniform_int_distribution<int> wdist(1, 30000);

    set<pair<int,int>> used;
    vector<Edge> edges;
    edges.reserve(m);

    for (int i = 2; i <= n; i++) {
        uniform_int_distribution<int> vdist(1, i - 1);
```

```
        int u = vdist(gen);
        int a = min(u, i), b = max(u, i);
        used.insert({a, b});
        edges.push_back({u, i, wdist(gen)});
    }

    uniform_int_distribution<int> vdist(1, n);
    while ((int)edges.size() < m && (int)used.size() < maxEdges) {
        int u = vdist(gen);
        int v = vdist(gen);
        if (u == v) continue;
        int a = min(u, v), b = max(u, v);
        if (used.count({a, b})) continue;
        used.insert({a, b});
        edges.push_back({u, v, wdist(gen)});
    }

    ofstream out("input.txt");
    if (!out) {
        cerr << "Не вдалося створити input.txt\n";
        return;
    }
    out << n << " " << edges.size() << "\n";
    for (const auto& e : edges) {
        out << e.u << " " << e.v << " " << e.w << "\n";
    }
    out.close();
}

int main(int argc, char* argv[]) {
    if (argc >= 3) {
        int n = atoi(argv[1]);
        int m = atoi(argv[2]);
        generateRandomGraph(n, m);
        cout << "Згенеровано input.txt: N=" << n << ", M=" << m << "\n";
        return 0;
    }

    ifstream in("input.txt");
    if (!in) {
        cout << "Файл input.txt не знайдено. Генеруємо випадковий граф...\n";
        generateRandomGraph(8, 15);
        in.open("input.txt");
        if (!in) {
            cerr << "Не вдалося створити input.txt\n";
            return 1;
        }
    }

    int n, m;
    if (!(in >> n >> m)) {
        cerr << "Помилка читання N, M\n";
        return 1;
    }

    vector<Edge> edges;
    edges.reserve(m);
    for (int i = 0; i < m; i++) {
        Edge e;
```

```
        if (!(in >> e.u >> e.v >> e.w)) {
            cerr << "Помилка читання ребра " << i + 1 << "\n";
            return 1;
        }
        edges.push_back(e);
    }
    in.close();

    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.w < b.w;
    });

    DSU dsu(n);

    long long total = 0;
    int used = 0;

    for (const auto& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            total += e.w;
            if (++used == n - 1)
                break;
        }
    }

    ofstream out("output.txt");
    if (!out) {
        cerr << "Не вдалося створити output.txt\n";
        return 1;
    }
    out << total << "\n";
    out.close();

    cout << "Вага мінімального кістяка: " << total << "\n";
    return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

Вивід програми:

```
1.  PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> g++ -O2 -std=c++20 main.cpp -o
    main.exe

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> Get-Content input.txt
3 3
1 2 1
2 3 2
3 1 3

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> ./main.exe
Вага мінімального кістяка: 3

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> Get-Content output.txt
3

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> ./main.exe 8 15
Згенеровано input.txt: N=8, M=15

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> Get-Content input.txt
8 15
1 2 26705
1 3 3922
1 4 1193
1 5 24794
3 6 15963
4 7 28690
7 8 13858
6 8 22468
2 6 26771
8 1 252
4 3 24219
7 5 28940
7 6 1515
4 2 21891
5 2 10912

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> ./main.exe
Вага мінімального кістяка: 53543

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> Get-Content output.txt
53543

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> ./main.exe 100 6000
Згенеровано input.txt: N=100, M=6000

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> ./main.exe
Вага мінімального кістяка: 34080

PS C:\Github\Learning-assistent\ALG_AND_STR\Labs\LAB7> Get-Content output.txt
34080
```

БЛОК-СХЕМА АЛГОРИТМУ КРАСКАЛА

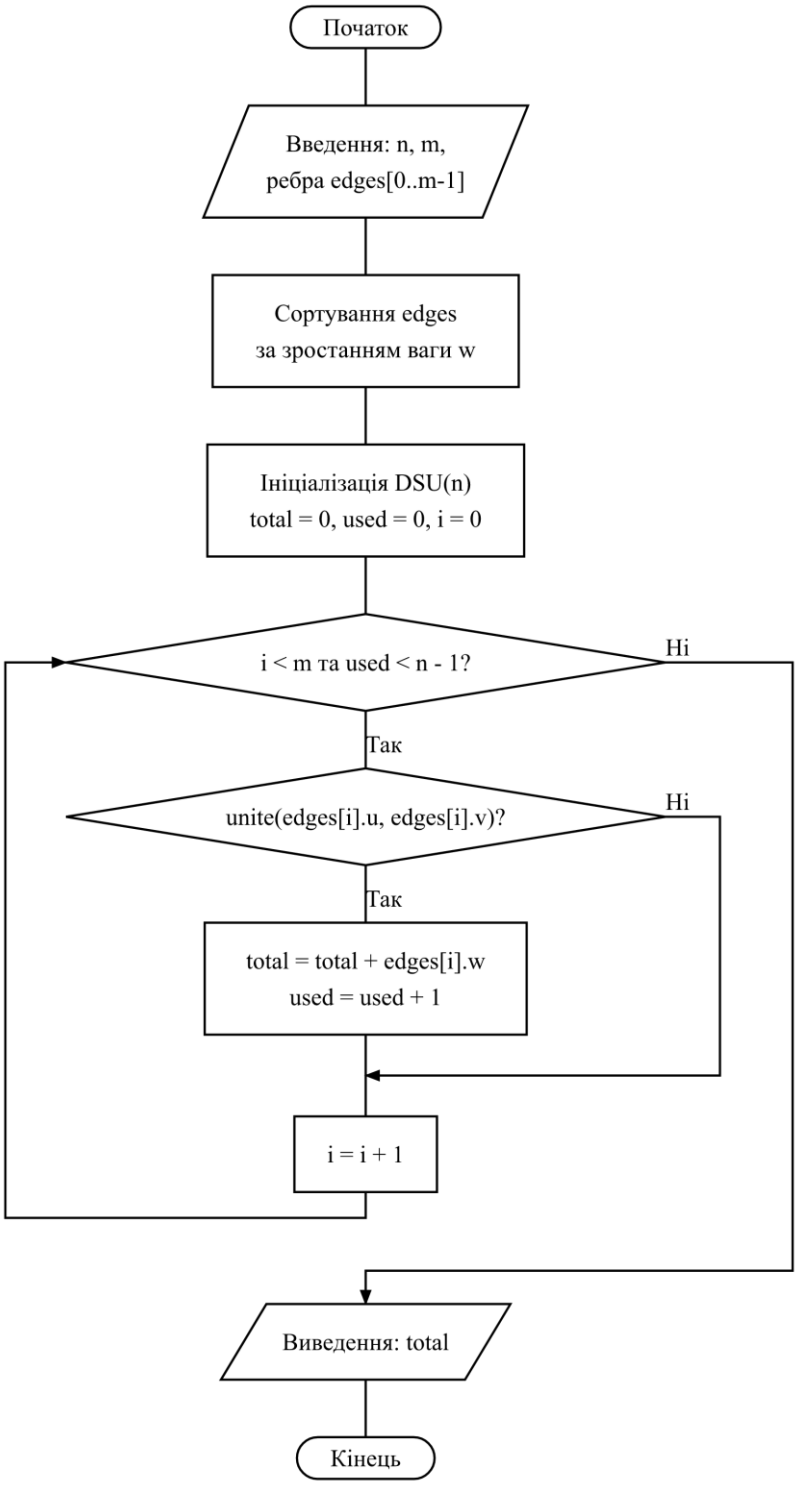


Рисунок 1 — Пошук ваги мінімального кістяка графа за алгоритмом Краскала

Вінниця 2026

БЛОК-СХЕМА ОБ'ЄДНАННЯ МНОЖИН У DSU

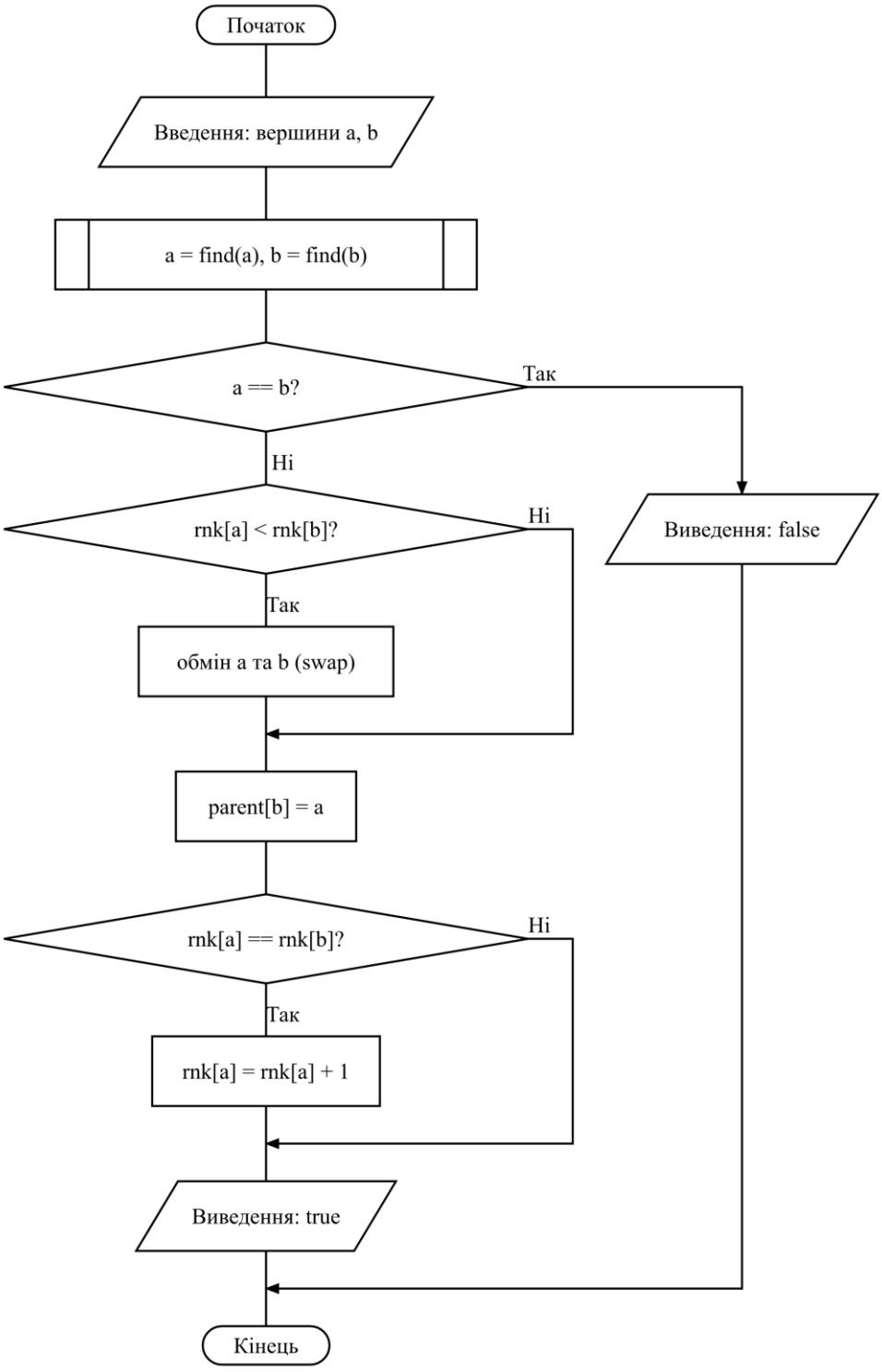


Рисунок 2 — Пошук представника та об'єднання множин у структурі DSU

Вінниця 2026

БЛОК-СХЕМА ГЕНЕРАЦІЇ ВИПАДКОВОГО ГРАФА

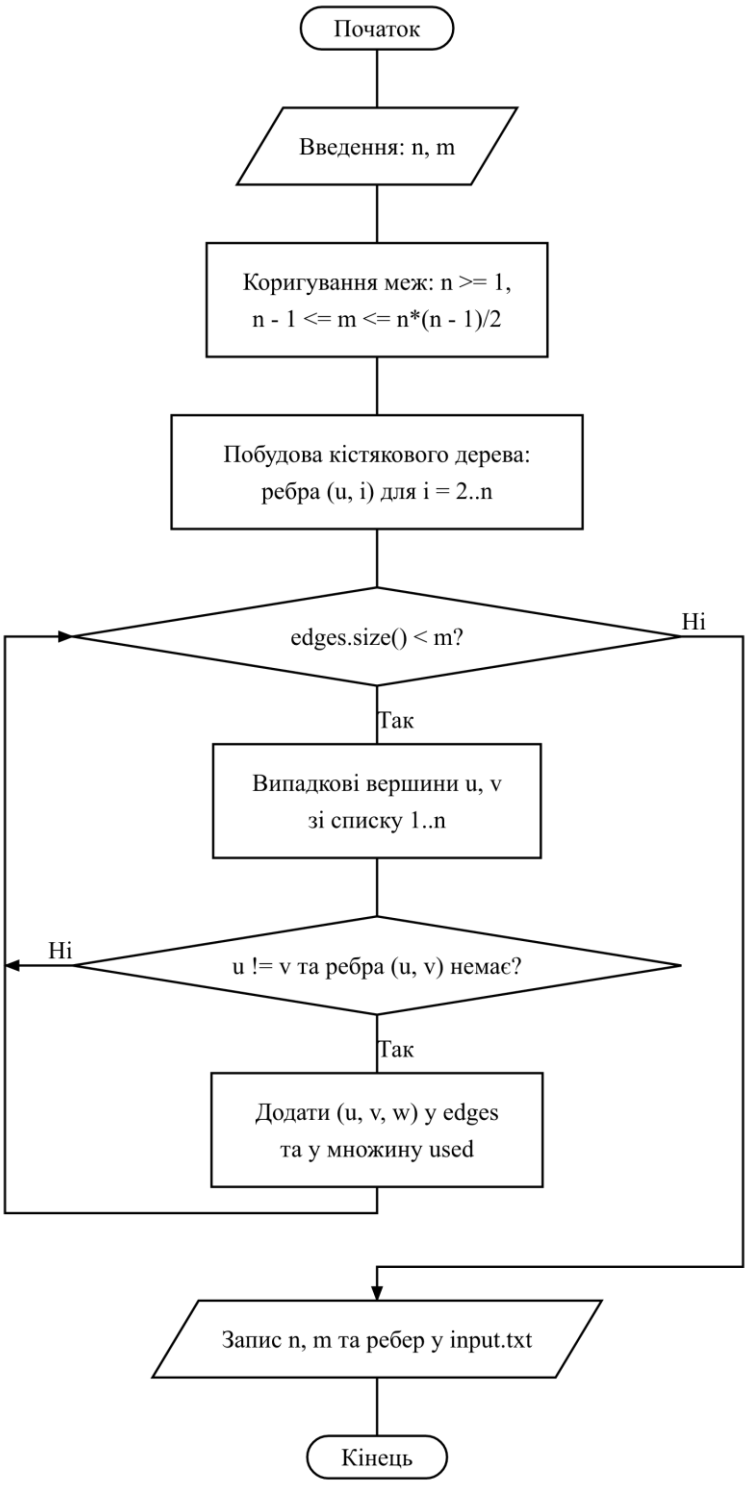


Рисунок 3 — Генерація випадкового зв'язкового зваженого графа

Вінниця 2026

Висновок: У результаті виконання лабораторної роботи досліджено алгоритми на графах та реалізовано мовою C++ програму для знаходження ваги мінімального кістяка (MST) неорієнтованого зваженого зв'язкового графа (варіант 12) за алгоритмом Краскала. Для перевірки належності вершин до однієї компоненти зв'язності та запобігання утворенню циклів реалізовано систему неперетинних множин (DSU) зі стисненням шляху та об'єднанням за рангом. Часова складність алгоритму становить O(M log M), а просторова — O(N + M), що забезпечує високу швидкодію при максимальних обмеженнях задачі (N = 100, M = 6000). Правильність роботи програми підтверджено тестуванням на контрольному прикладі (вага кістяка 3) та на згенерованих випадкових зв'язкових графах.

Вінниця 2026